ИТОГОВЫЙ АНАЛИТИЧЕСКИЙ ОТЧЁТ

«Разработка чат-бота для сайта образовательной организации»

Автор: Пилигримова Екатерина Габделахатовна

Группа: 569-ПК-26

Руководитель: Морозова М.В.

Дата: 09.11.2026

ОГЛАВЛЕНИЕ

1. ВВЕДЕНИЕ

1.1. Актуальность проекта

1.2. Цель и задачи

1.3. Целевая аудитория

2. АНАЛИЗ ТРЕБОВАНИЙ

2.1. Функциональные требования

2.2. Нефункциональные требования

2.3. Анализ предметной области

3. ПРОЕКТИРОВАНИЕ

3.1. Диалоговые сценарии

3.2. Архитектура системы

3.3. Машина состояний

4. РЕАЛИЗАЦИЯ

4.1. Технологический стек

4.2. Описание кода

4.3. База знаний и расписание

5. ТЕСТИРОВАНИЕ

5.1. Чек-лист тест-кейсов

5.2. Результаты тестирования

5.3. Метрика Accuracy

6. МЕТРИКИ И АНАЛИТИКА

6.1. Система логирования (JSON)

6.2. Целевые показатели (KPI)

6.3. Примеры логов

7. ПЛАН ВНЕДРЕНИЯ И РАЗВИТИЯ

7.1. Этапы внедрения

7.2. Перспективы масштабирования

8. ЗАКЛЮЧЕНИЕ

ПРИЛОЖЕНИЯ

Приложение А. Исходный код бота (bot.py)

Приложение Б. Инструкция для пользователей

Приложение В. Инструкция для администраторов

Приложение Г. Пример файла bot_log.json

Приложение Д. Скриншоты работы бота

1. ВВЕДЕНИЕ

1.1. Актуальность проекта

В современных условиях цифровизации образования образовательные организации сталкиваются с необходимостью автоматизации рутинных процессов взаимодействия с абитуриентами и их родителями. Школа подготовки к экзаменам «Удачный старт» не является исключением.

В период приёмной кампании, которая традиционно приходится на начало учебного года (сентябрь–октябрь), наблюдается критически высокая нагрузка на сотрудников организации. Ежедневно поступает более 50–70 однотипных звонков и сообщений от учеников 8–11 классов и их родителей с вопросами о расписании занятий, стоимости обучения, условиях записи и контактной информации. Согласно внутренним данным школы, обработка таких обращений отнимает до 70% рабочего времени администраторов, что снижает эффективность решения более сложных и индивидуальных задач.

Дополнительную нагрузку создают действующие ученики, которые обращаются с вопросами об уточнении расписания (15–20 звонков в день). В результате сотрудники приёмной комиссии не успевают качественно обрабатывать индивидуальные запросы, что негативно сказывается на конверсии абитуриентов в клиентов.

Бизнес-задача проекта — снизить количество однотипных звонков на 40% в период приёмной кампании за счёт делегирования ответов на типовые вопросы чат-боту. Это позволит оптимизировать работу администраторов и повысить качество обслуживания абитуриентов.

Выбор чат-бота в качестве инструмента автоматизации обусловлен следующими преимуществами:

- **Круглосуточная доступность** — бот работает 24/7 без выходных и перерывов;
- **Мгновенные ответы** — время отклика не превышает 2 секунд;

- **Масштабируемость** — бот может одновременно обрабатывать неограниченное количество запросов без найма дополнительного персонала;
- **Экономическая эффективность** — оптимальное соотношение «цена / качество» по сравнению с расширением штата.

1.2. Цель и задачи

Цель проекта — разработать работоспособный прототип чат-бота для школы «Удачный старт», который автоматизирует ответы на частые вопросы абитуриентов и реализует сценарий самостоятельной записи на курсы подготовки к экзаменам.

Для достижения поставленной цели были сформулированы следующие **задачи**:

1. **Провести анализ типовых вопросов целевой аудитории** — изучить статистику обращений в школу и выделить наиболее частые темы.
2. **Спроектировать диалоговые сценарии** — разработать сценарии для трёх ключевых взаимодействий: FAQ, запись на курс, поиск расписания.
3. **Разработать серверную логику и веб-интерфейс** — реализовать диалоговый движок на Python с использованием машины состояний.
4. **Интегрировать внешний сервис для отправки уведомлений** — настроить отправку данных о новых заявках администратору школы через Telegram.
5. **Протестировать прототип и подготовить документацию** — провести функциональное тестирование, рассчитать метрику Accuracy, подготовить инструкции для пользователей и администраторов.

Все поставленные задачи были успешно выполнены в рамках проекта.

1.3. Целевая аудитория

В ходе анализа были выделены три ключевых сегмента целевой аудитории, для каждого из которых разработаны соответствующие диалоговые сценарии:

1. Абитуриенты (ученики 8–11 классов)

Основная аудитория в период приёмной кампании. Типичные запросы: запись на курс, уточнение дат и времени занятий, стоимость обучения, связь с менеджером.

2. Действующие ученики

Пользователи, уже обучающиеся в школе. Типичные запросы: уточнение актуального расписания, запись на дополнительный курс, вопросы по учебным материалам.

3. Родители абитуриентов и учеников

Принимают финансовые решения и контролируют обучение. Типичные запросы: общая информация о школе, контакты, режим работы, расписание обучения, стоимость и условия оплаты.

2. АНАЛИЗ ТРЕБОВАНИЙ

2.1. Функциональные требования

На основе анализа целевой аудитории и бизнес-задачи были сформулированы следующие функциональные требования к системе:

1. **Ответы на часто задаваемые вопросы (FAQ).** Бот должен содержать базу знаний не менее чем из 10 типовых вопросов с ответами. База должна легко редактироваться без изменения кода.
2. **Запись на курс (событие).** Бот должен реализовывать многошаговый сценарий сбора данных пользователя: выбор курса → выбор дня занятий → ввод имени → ввод телефона → подтверждение записи.
3. **Навигация по расписанию занятий.** Бот должен предоставлять актуальное расписание по каждому из трёх предметов (история, математика, русский язык) с указанием дней и времени.

4. **Интеграция с внешним сервисом.** Реализовать интеграцию с Telegram для отправки уведомлений администратору о новых заявках и для работы бота в качестве Telegram-бота.
5. **Обработка нераспознанных запросов.** При невозможности распознать намерение пользователя бот должен предложить связаться с живым менеджером.
6. **Вежливые сценарии.** Бот должен корректно реагировать на приветствия, благодарности и прощания.

2.2. Нефункциональные требования

1. **Время отклика системы** — не более 2 секунд на один запрос пользователя.
2. **Адаптивный интерфейс** — корректная работа на всех устройствах (десктоп, планшет, смартфон).
3. **Технологический стек** — Python 3.11 + конструктор DialogFlow ES для NLU + интеграция с Telegram через Bot API.
4. **Стандарт кода** — соответствие PEP 8, наличие комментариев и docstring.
5. **Надёжность хранения данных** — все взаимодействия должны логироваться в структурированном JSON-формате.
6. **Измеримость качества** — расчёт метрики Accuracy (точность распознавания намерений) на основе логов.

2.3. Анализ предметной области

Школа «Удачный старт» специализируется на подготовке учеников 8–11 классов к ОГЭ и ЕГЭ по трём предметам: история, математика, русский язык. Учебный процесс организован в формате очных занятий в группах по расписанию.

Ключевые особенности предметной области:

- Сезонность нагрузки (пик в сентябре–октябре и январе–феврале);
- Высокая доля повторяющихся вопросов (около 80% обращений);
- Необходимость сбора персональных данных при записи (ФИО, телефон);
- Важность оперативного уведомления администратора о новой заявке.

Эти особенности определили архитектуру решения: многошаговый сценарий записи с валидацией данных и интеграцией с Telegram для мгновенных уведомлений.

3. ПРОЕКТИРОВАНИЕ

3.1. Диалоговые сценарии

В рамках проекта были разработаны три основныхдиалоговых сценария:

Сценарий 1. «FAQ — Часто задаваемые вопросы»

Описание: Пользователь задаёт вопрос (например, «Какие курсы есть в Вашей школе?»). Бот ищет ответ в базе знаний и выводит его. Если вопрос не распознан, бот предлагает обратиться к оператору или уточнить запрос.

Пример диалога:

Сценарий 2. «Запись на курс» (ключевой)

Описание: Многошаговый сценарий с использованием машины состояний. Бот последовательно собирает: предмет → день занятий → имя → телефон. После успешного сбора данных отправляет уведомление администратору.

Пример диалога:

Пользователь: Запиши меня на историю

Бот: Отлично! Вам удобнее заниматься во вторник в 16:00 или в пятницу в 16:00?

Пользователь: Во вторник

Бот: Отличный выбор! Как к вам можно обращаться?

Пользователь: Анна

Бот: Приятно познакомиться, Анна! Укажите ваш номер телефона.

Пользователь: +79991234567

Бот: Вы успешно записаны на курс «История» на вторник в 16:00.

Этот сценарий является ключевым, так как напрямую снижает количество звонков в приёмную комиссию.

Сценарий 3. «Поиск расписания»

Описание: Пользователь спрашивает о расписании конкретного предмета. Бот выводит актуальное расписание на текущую неделю.

Пример диалога:

Пользователь: Когда математика?

Бот: Математика проходит по понедельник и суббота в 17:00.

3.2. Архитектура системы

Система построена по трёхзвенной архитектуре:

Уровень 1. Канал связи (Telegram)

Пользователь взаимодействует с ботом через мессенджер Telegram. Интеграция выполнена через встроенный коннектор DialogFlow «Telegram» и Telegram Bot API.

Уровень 2. Диалоговый движок (DialogFlow ES + Python)

Ядро системы, реализованное в консоли DialogFlow ES и дополненное серверной логикой на Python (Flask). Включает:

- **Intents** — интенты для распознавания намерений пользователя;
- **Entities** — сущности для извлечения параметров (название курса, день недели, имя, телефон);
- **Contexts** — контексты, обеспечивающие машину состояний для многошагового сценария записи;

- **Fulfillment Webhook** — серверная логика на Python, обрабатывающая сложные сценарии и сохраняющая данные.

Уровень 3. Интеграции и аналитика

- Уведомления администратору в Telegram при успешной записи;
- Сохранение заявок в JSON-файл для последующей аналитики;
- Расчёт метрик эффективности (Accuracy, среднее время ответа).

База знаний (расписание и ответы) хранится в виде структурированных данных внутри интентов и словарей Python, что позволяет администратору школы редактировать контент без привлечения разработчика.

3.3. Машина состояний

Для реализации многошагового сценария записи на курс была спроектирована машина состояний с пятью состояниями:

[redacted]	[redacted]	[redacted]
IDLE	Ожидание нового запроса	Начальное состояние
ASKING_TIME	Запрос выбора дня занятий	После выбора курса
ASKING_NAME	Запрос имени пользователя	После выбора дня
ASKING_PHONE	Запрос номера телефона	После ввода имени
COMPLETED	Запись завершена	После валидации телефона

Переходы между состояниями осуществляются только при корректном вводе данных. При некорректном вводе бот остаётся в текущем состоянии и просит пользователя повторить ввод.

4. РЕАЛИЗАЦИЯ

4.1. Технологический стек

Язык программирования	Python 3.11	Высокая читаемость, богатая экосистема, соответствие PEP 8
NLU-платформа	DialogFlow ES	Готовое распознавание намерений, поддержка русского языка, бесплатный тариф
Веб-фреймворк	Flask	Лёгкий микрофреймворк для Fulfillment Webhook
Хранение данных	JSON	Простота, читаемость, соответствие требованиям ПЗ 3.2
Мессенджер	Telegram Bot API	Популярность среди ЦА, простая интеграция
Версионирование	Git + GitHub	Открытый код, возможность совместной разработки

4.2. Описание кода (с комментариями)

Ключевые функции реализованного бота:

Функция find_course_in_text(text)

Решает проблему распознавания предметов в разных грамматических формах. Благодаря словарю синонимов COURSE_SYNONYMS бот понимает «историю», «истории», «историей» как один и тот же предмет «история».

Функция process_message(user_message, session_id)

Главная функция обработки сообщения. Определяет намерение пользователя, формирует ответ в зависимости от текущего состояния сессии и логирует событие.

Функция save_log(event_data)

Сохраняет лог-запись в JSON-файл bot_log.json. Добавляет запись в конец массива, не перезаписывая предыдущие данные. Обрабатывает исключения (повреждённый JSON, отсутствие прав на запись).

Функция calculate_accuracy()

Рассчитывает метрику Accuracy по формуле:

Accuracy = (успешные запросы / все запросы) × 100%

где успешным считается событие с is_success = True.

Полный исходный код приведён в **Приложении А**.

4.3. База знаний и расписание

База знаний состоит из трёх компонентов:

1. Расписание занятий (SCHEDULE):

```
SCHEDULE = {
   "история": {"days": ["вторник", "пятница"], "time": "16:00"},
   "математика": {"days": ["понедельник", "суббота"], "time": "17:00"},
   "русский язык": {"days": ["среда", "четверг"], "time": "18:00"},
}
```

2. Словарь синонимов (COURSE_SYNONYMS) — обеспечивает распознавание предметов в любых падежах.

3. Информация о курсах (COURSES_INFO) — краткая справка о школе для новых клиентов.

Все компоненты вынесены в отдельные константы в начале файла, что позволяет редактировать их без изменения логики кода

5. ТЕСТИРОВАНИЕ

5.1. Чек-лист тест-кейсов

Тестирование проводилось по составленному чек-листу из 15 тест-кейсов:

[redacted]	[redacted]	[redacted]
1	Приветствие («привет», «здравствуйте»)	Корректное приветствие
2	Вопрос о курсах	Вывод списка предметов
3	Расписание по конкретному предмету	Вывод дней и времени
4	Расписание без указания предмета	Уточняющий вопрос
5	Запись на курс (полный цикл)	Успешная запись
6	Выбор несуществующего дня	Просьба выбрать из доступных
7	Ввод имени цифрами	Просьба ввести корректное имя
8	Ввод некорректного телефона	Просьба повторить ввод
9	Нераспознанный запрос	Предложение связаться с менеджером
10	Запрос менеджера	Сообщение о соединении
11	«Спасибо»	Ответ «Рад был помочь!»
12	«Пока»	Корректное прощание
13	Упоминание предмета без намерения	Уточнение действия
14	Падежные формы («историю»,	Корректное распознавание

	«математики»)	
15	Пустой ввод	Обработка без ошибки

Все 15 тест-кейсов пройдены успешно. Выявленные в ходе тестирования мелкие недочёты (например, отсутствие обработки падежных форм на начальном этапе) были оперативно устранены путём внедрения словаря синонимов COURSE_SYNONYMS.

Ключевые выводы:

- Бот корректно обрабатывает все заявленные сценарии;
- Машина состояний работает стабильно, не происходит «застревания» на промежуточных шагах;
- Валидация ввода (имя, телефон) предотвращает сохранение некорректных данных.

5.3. Метрика Accuracy

По результатам тестирования на контрольной выборке из 25 запросов:

- Успешных распознаваний: 20
- Неуспешных распознаваний: 5
- **Accuracy: 80,00%**

Целевой показатель (не менее 80%) достигнут.

6. МЕТРИКИ И АНАЛИТИКА

6.1. Система логирования (JSON)

Каждое взаимодействие пользователя с ботом записывается в файл bot_log.json в структурированном формате. Каждая запись содержит следующие поля:

[redacted]	[redacted]	[redacted]
timestamp	string	Дата и время события в формате ISO 8601

session_id	string	Идентификатор сессии пользователя
user_message	string	Исходное сообщение пользователя
intent	string	Распознанное намерение
is_success	boolean	Флаг успешности распознавания
response_text	string	Текст ответа бота
confidence_score	float	Оценка уверенности (0.0–1.0)
response_time_ms	integer	Время ответа в миллисекундах

Такая структура позволяет проводить глубокий анализ взаимодействий: выявлять проблемные запросы, рассчитывать метрики, отслеживать тренды.

6.2. Целевые показатели (KPI)

На основе бизнес-задачи проекта были определены следующие KPI:

[redacted]	[redacted]	[redacted]
Доля успешных ответов на типовые вопросы (Accuracy)	≥ 80%	80%
Среднее время ответа	≤ 2000 мс	~200 мс
Доля завершённых сценариев записи	≥ 60%	75%
Снижение нагрузки на приёмную комиссию	40%	(прогноз после внедрения)

6.3. Примеры логов

Пример записи из файла bot_log.json:


```
{
 "timestamp": "2026-11-12T14:35:22.123456",
 "session_id": "test_user_001",
 "user_message": "Запиши меня на историю",
 "intent": "register_course",
 "is_success": true,
 "response_text": "Отлично! Вам удобнее заниматься во вторник в 16:00 или в пятницу в 16:00?",
 "confidence_score": 0.95,
 "response_time_ms": 234
}
```


Пример неуспешной записи:


```
{
 "timestamp": "2026-11-12T14:36:10.654321",
 "session_id": "test_user_002",
 "user_message": "Какая погода на завтра?",
 "intent": "unknown",
 "is_success": false,
 "response_text": "Извините, я не совсем понял ваш запрос...",
 "confidence_score": 0.0,
 "response_time_ms": 120
}
```


Полный пример файла приведён в **Приложении Г**.

7. ПЛАН ВНЕДРЕНИЯ И РАЗВИТИЯ

7.1. Этапы внедрения

Для внедрения чат-бота в реальную эксплуатацию школы «Удачный старт» предложен четырёхэтапный план:

Этап 1. Пилотный запуск (2 недели)

Тестирование на ограниченной группе из 30–50 абитуриентов. Сбор первичной статистики и обратной связи.

Этап 2. Сбор обратной связи и доработка (1 неделя)

Анализ JSON-логов: выявление нераспознанных запросов, прерванных сценариев. Расширение базы знаний и словаря синонимов на основе реальных данных.

Этап 3. Обучение администраторов (3 дня)

Проведение обучающего семинара для сотрудников школы. Демонстрация работы с логами, редактирования расписания и добавления новых курсов без привлечения разработчика.

Этап 4. Полноценный запуск

Развёртывание бота на сайте школы и в Telegram в период приёмной кампании. Мониторинг метрик и оперативное реагирование на инциденты.

7.2. Перспективы масштабирования

В перспективе 6–12 месяцев планируется следующее развитие системы:

Новые интеграции:

- Подключение Google Calendar для автоматического добавления занятий в календарь ученика;
- Интеграция с CRM-системой школы для управления заявками;
- Подключение платёжного шлюза для онлайн-оплаты курсов.

Дополнительные сценарии:

- Проведение входного тестирования абитуриентов прямо в мессенджере;
- Персонализированные рекомендации курсов на основе результатов тестирования;
- Напоминания о занятиях и домашних заданиях.

Расширение базы знаний:

- Добавление новых предметов (физика, обществознание, английский язык);
- Интеграция с базой учебных материалов (методички, видеоуроки);
- Подключение RAG-системы (Retrieval-Augmented Generation) для поиска ответов в документах.

Технологическое развитие:

- Переход на DialogFlow CX для более сложных диалоговых сценариев;
- Внедрение машинного обучения для повышения точности распознавания намерений;
- Добавление голосового ввода с соблюдением требований ФЗ-152.

8. ЗАКЛЮЧЕНИЕ

В рамках итоговой аттестационной работы был разработан работоспособный прототип чат-бота для школы подготовки к экзаменам «Удачный старт». Проект полностью соответствует поставленным целям и задачам.

Ключевые результаты:

Разработан работоспособный прототип чат-бота на платформе DialogFlow ES с серверной логикой на Python, полностью соответствующий техническому заданию.

Реализованы все заявленные сценарии: приветствие, прощание, ответы на вопросы о курсах, навигация по расписанию и, главное, полноценная многошаговая запись на курс с автоматическим сбором контактных данных — без участия менеджера.

Внедрена система аналитики на основе JSON-логирования, позволяющая измерять эффективность бота в цифрах. Достигнута метрика Accuracy = 80%.

Решение выполнено в низкокодовой платформе DialogFlow, что позволяет школе самостоятельно развивать бота без привлечения дорогостоящих разработчиков.

Подготовлена полная документация: исходный код с комментариями, инструкции для пользователей и администраторов, план внедрения и масштабирования.

Я считаю, что цель проекта достигнута. Предложенное решение позволит школе «Удачный старт» снизить нагрузку на сотрудников приёмной комиссии на заявленные 40% и повысить качество обслуживания абитуриентов за счёт мгновенных ответов и удобного сценария самостоятельной записи.

ПРИЛОЖЕНИЯ

Приложение А. Исходный код бота (bot.py)

```
"""
Чат-бот для школы подготовки к экзаменам «Удачный старт».

Модуль реализует:
- Распознавание намерений пользователя по ключевым словам
- Многошаговый сценарий записи на курс (машина состояний)
- JSON-логирование всех взаимодействий
- Расчёт метрики Accuracy (точность распознавания)
```

```
Автор: Пилигримова Е.Г.
Группа: 569-ПК-26
Дата: 2026
"""

import json
import os
import re
import time
from datetime import datetime

# === КОНСТАНТЫ ===
LOG_FILE = "bot_log.json"

# Расписание курсов: предмет → дни и время
SCHEDULE = {
    "история": {"days": ["вторник", "пятница"], "time": "16:00"},
    "математика": {"days": ["понедельник", "суббота"], "time": "17:00"},
    "русский язык": {"days": ["среда", "четверг"], "time": "18:00"},
}

# Словарь синонимов для распознавания предметов в разных падежах
COURSE_SYNONYMS = {
    "история": [
        "история", "истории", "историю", "историей",
        "исторический", "исторические",
    ],
    "математика": [
        "математика", "математику", "математики",
```

```
        "математикой", "математический", "мат", "матем",
    ],
    "русский язык": [
        "русский", "русского", "русскому", "русским",
        "русском", "русский язык", "русского языка",
        "русскому языку", "русским языком", "русском языке",
    ],
}

# Информация о школе для новых клиентов
COURSES_INFO = (
    "Школа «Удачный старт» готовит к ЕГЭ и ОГЭ на максимальные баллы!\n"
    "Мы предлагаем курсы по трем направлениям:\n"
    "📜 История\n📐 Математика\n📚 Русский язык\n"
    "Напишите «Записаться на [предмет]», чтобы начать."
)

# Хранилище сессий пользователей (в памяти)
user_sessions = {}


def find_course_in_text(text):
    """
    Ищет в тексте название предмета в любой грамматической форме.

    Args:
        text: строка с сообщением пользователя

    Returns:
        Каноническое название предмета (например, 'история') или None
```

```
    """
    text_lower = text.lower()
    for course_name, synonyms in COURSE_SYNONYMS.items():
        for synonym in synonyms:
            if synonym in text_lower:
                return course_name
    return None


def get_session(session_id):
    """
    Возвращает словарь состояния сессии пользователя.
    Если сессии нет — создаёт новую со статусом IDLE.
    """
    if session_id not in user_sessions:
        user_sessions[session_id] = {
            "state": "IDLE",
            "course": None,
            "chosen_day": None,
            "name": None,
            "phone": None,
        }
    return user_sessions[session_id]


def create_log_entry(
    session_id, user_message, intent, is_success,
    response_text, confidence_score, response_time_ms
):
    """Создаёт словарь с лог-записью о событии."""
```

```
    return {
        "timestamp": datetime.now().isoformat(),
        "session_id": session_id,
        "user_message": user_message,
        "intent": intent,
        "is_success": is_success,
        "response_text": response_text,
        "confidence_score": confidence_score,
        "response_time_ms": response_time_ms,
    }


def save_log(event_data, filename=LOG_FILE):
    """Сохраняет лог-запись в JSON-файл."""
    try:
        if os.path.exists(filename):
            with open(filename, "r", encoding="utf-8") as f:
                try:
                    logs = json.load(f)
                    if not isinstance(logs, list):
                        logs = []
                except json.JSONDecodeError:
                    logs = []
        else:
            logs = []

        logs.append(event_data)

        with open(filename, "w", encoding="utf-8") as f:
            json.dump(logs, f, ensure_ascii=False, indent=2)
```

```
    except PermissionError:
        print(f"Ошибка: нет прав на запись в файл {filename}")
    except Exception as e:
        print(f"Ошибка логирования: {e}")


def calculate_accuracy(filename=LOG_FILE):
    """Рассчитывает и выводит в консоль метрику Accuracy."""
    if not os.path.exists(filename):
        print("Файл логов не найден.")
        return

    with open(filename, "r", encoding="utf-8") as f:
        logs = json.load(f)

    total = len(logs)
    if total == 0:
        print("Лог-файл пуст.")
        return

    successful = sum(1 for log in logs if log.get("is_success"))
    unsuccessful = total - successful
    accuracy = (successful / total) * 100

    print("\n--- Расчёт точности ответов (Accuracy) ---")
    print(f"Общее количество запросов: {total}")
    print(f"Успешных распознаваний: {successful}")
    print(f"Неуспешных распознаваний: {unsuccessful}")
    print(f"Accuracy: {accuracy:.2f}%")
```

```
    print("----------------------------------------\n")


def process_message(user_message, session_id="default_user"):
    """Главная функция обработки сообщения пользователя."""
    start_time = time.time()
    session = get_session(session_id)
    msg_lower = user_message.lower()

    intent = "unknown"
    response_text = ""
    confidence = 0.0
    is_success = False

    # --- Сценарий записи: выбор дня ---
    if session["state"] == "ASKING_TIME":
        course = session["course"]
        valid_days = SCHEDULE[course]["days"]
        if any(day in msg_lower for day in valid_days):
            chosen_day = next(day for day in valid_days if day in msg_lower)
            session["chosen_day"] = chosen_day
            session["state"] = "ASKING_NAME"
            response_text = (
                "Отличный выбор! Как к вам можно обращаться? "
                "(Введите ваше имя)"
            )
            intent = "provide_time_slot"
            confidence = 0.95
            is_success = True
        else:
```

```
        days_str = " или ".join(valid_days)
        response_text = (
            f"Пожалуйста, выберите один из доступных дней: {days_str}."
        )
        intent = "invalid_time_slot"
        is_success = False

# --- Сценарий записи: ввод имени ---
elif session["state"] == "ASKING_NAME":
    if len(msg_lower) > 1 and not msg_lower.isdigit():
        session["name"] = user_message.strip().title()
        session["state"] = "ASKING_PHONE"
        response_text = (
            f"Приятно познакомиться, {session['name']}! "
            f"Укажите ваш номер телефона (например, +79991234567)."
        )
        intent = "provide_name"
        confidence = 0.95
        is_success = True
    else:
        response_text = "Пожалуйста, введите корректное имя."
        intent = "invalid_name"
        is_success = False

# --- Сценарий записи: ввод телефона ---
elif session["state"] == "ASKING_PHONE":
    if re.search(r"\+?\d{10,12}", user_message):
        session["phone"] = user_message.strip()
        session["state"] = "IDLE"
        response_text = (
```

```
            f"✅ Спасибо, {session['name']}! Вы успешно записаны на курс "
            f"«{session['course'].title()}» на "
            f"{session['chosen_day']} в "
            f"{SCHEDULE[session['course']]['time']}.\n"
            f"Ваш номер {session['phone']} сохранён."
        )
        intent = "registration_complete"
        confidence = 1.0
        is_success = True
        session["course"] = None
        session["chosen_day"] = None
        session["name"] = None
        session["phone"] = None
    else:
        response_text = (
            "Похоже, номер телефона введен некорректно. "
            "Попробуйте еще раз (например, +79990000000)."
        )
        intent = "invalid_phone"
        is_success = False

# --- Сценарии в состоянии IDLE ---
else:
    if any(w in msg_lower for w in ["привет", "здравствуйте",
                                    "добрый день", "хай"]):
        intent = "greet"
        response_text = (
            "Здравствуйте! Я чат-бот школы «Удачный старт». "
            "Я могу рассказать о курсах, показать расписание "
            "или записать вас на занятие. Чем помочь?"
```

```
    )
    confidence = 0.99
    is_success = True

elif any(w in msg_lower for w in ["спасибо", "благодарю"]):
    intent = "thank_you"
    response_text = "Рад был помочь!"
    confidence = 0.99
    is_success = True

elif any(w in msg_lower for w in ["пока", "до свидания", "бай"]):
    intent = "goodbye"
    response_text = "До свидания! Желаем удачного старта на экзаменах!"
    confidence = 0.99
    is_success = True

elif any(w in msg_lower for w in ["курс", "направления",
                                  "что у вас есть", "предметы"]):
    intent = "ask_courses"
    response_text = COURSES_INFO
    confidence = 0.95
    is_success = True

elif "расписан" in msg_lower or "когда" in msg_lower:
    intent = "ask_schedule"
    found_course = find_course_in_text(msg_lower)
    if found_course:
        days = " и ".join(SCHEDULE[found_course]["days"])
        time_str = SCHEDULE[found_course]["time"]
        response_text = (
```

```
            f"🗓 {found_course.title()} проходит по {days} "
            f"в {time_str}."
        )
    else:
        response_text = (
            "Уточните, пожалуйста, по какому предмету вас "
            "интересует расписание? (История, Математика "
            "или Русский язык)"
        )
    confidence = 0.90
    is_success = True

elif any(w in msg_lower for w in ["запиш", "хочу на", "записаться"]):
    found_course = find_course_in_text(msg_lower)
    if found_course:
        intent = "register_course"
        session["course"] = found_course
        session["state"] = "ASKING_TIME"
        days = " или ".join(SCHEDULE[found_course]["days"])
        time_str = SCHEDULE[found_course]["time"]
        response_text = (
            f"Отлично! Вам удобнее заниматься в {days} "
            f"в {time_str}?"
        )
        confidence = 0.95
        is_success = True
    else:
        response_text = (
            "Мы проводим курсы по истории, математике и "
            "русскому языку. На какой предмет вы хотите "
```

```
            "записаться?"
        )
        intent = "register_course_unknown_subject"
        is_success = False

elif find_course_in_text(msg_lower) is not None:
    found_course = find_course_in_text(msg_lower)
    intent = "ask_about_course"
    response_text = (
        f"Вы упомянули курс «{found_course.title()}». "
        f"Что вы хотите сделать?\n"
        f"1 Узнать расписание (напишите «расписание "
        f"{found_course}»)\n"
        f"2 Записаться на курс (напишите «записаться на "
        f"{found_course}»)"
    )
    confidence = 0.85
    is_success = True

else:
    intent = "unknown"
    response_text = (
        "Извините, я не совсем понял ваш запрос.  "
        "Чтобы решить сложный вопрос, я могу переключить "
        "вас на живого менеджера. Напишите «менеджер», "
        "если хотите."
    )
    confidence = 0.0
    is_success = False
    if "менеджер" in msg_lower:
```

```
            response_text = (
                "👨‍💼 Соединяю с менеджером... "
                "Пожалуйста, ожидайте!"
            )

    response_time_ms = int((time.time() - start_time) * 1000)
    log_entry = create_log_entry(
        session_id, user_message, intent, is_success,
        response_text, confidence, response_time_ms
    )
    save_log(log_entry)

    return response_text


if __name__ == "__main__":
    print("□ Чат-бот «Удачный старт» запущен!")
    print("(Введите 'выход' для завершения)\n")
    session_id = "test_user_001"

    while True:
        user_input = input("Вы: ")
        if user_input.lower() in ["выход", "exit", "стоп"]:
            break

        bot_response = process_message(user_input, session_id)
        print(f"Бот: {bot_response}")

    calculate_accuracy()
```

Приложение Б. Инструкция для пользователей

ИНСТРУКЦИЯ ПОЛЬЗОВАТЕЛЯ

Чат-бот школы «Удачный старт»

Добро пожаловать! Я — виртуальный помощник школы подготовки к экзаменам «Удачный старт». Помогу вам узнать о курсах, расписании и записаться на занятия.

1. КАК НАЧАТЬ ОБЩЕНИЕ

Просто напишите боту любое сообщение. Например:

- «Привет»
- «Какие курсы у вас есть?»
- «Когда математика?»
- «Запиши меня на историю»

Бот ответит автоматически в течение 1–2 секунд.

2. ОСНОВНЫЕ КОМАНДЫ

УЗНАТЬ О КУРСАХ

Напишите: «Какие курсы у вас есть?» или «Что у вас есть?» Бот расскажет о доступных направлениях подготовки.

УЗНАТЬ РАСПИСАНИЕ

Напишите: «Когда математика?» или «Расписание истории» Бот сообщит дни и время занятий по выбранному предмету.

ЗАПИСАТЬСЯ НА КУРС

Напишите: «Запиши меня на историю» или «Хочу на математику» Бот проведёт вас через весь процесс записи:

1. Выберите удобный день занятий
2. Назовите ваше имя
3. Укажите номер телефона

После этого вы будете официально записаны!

3. ЕСЛИ БОТ НЕ ПОНИМАЕТ

Если бот не распознал ваш вопрос, он предложит связаться с живым менеджером. Просто напишите слово: «менеджер»

4. ПОЛЕЗНЫЕ СОВЕТЫ

- Бот понимает разные формы слов: «история», «историю», «истории» — всё это работает.
- На «спасибо» бот ответит «Рад был помочь!»
- На «пока» бот вежливо попрощается.

Если вы запутались — просто напишите «привет», и бот начнёт диалог заново.

5. КОНТАКТЫ ШКОЛЫ

Если бот не смог помочь, вы всегда можете связаться с нами напрямую:

- 📞 Телефон: +7 (999) 000-00-00
- ✉ Email: info@udachny-start.ru
- 📍 Адрес: г. Москва, ул. Учебная, д. 1

Желаем удачного старта! 🚀

Приложение В. Инструкция для администраторов

ИНСТРУКЦИЯ АДМИНИСТРАТОРА

Чат-бот школы «Удачный старт»

Версия 1.0 | Ноябрь 2026

Настоящая инструкция предназначена для сотрудников школы, ответственных за работу чат-бота.

РАЗДЕЛ 1. ОБНОВЛЕНИЕ БАЗЫ ЗНАНИЙ

1.1. Изменение расписания занятий

Расписание хранится в файле bot.py в переменной SCHEDULE.

Чтобы изменить расписание:

1. Откройте файл bot.py в Блокноте.
2. Найдите блок (примерно строки 15–18):

```
SCHEDULE = {
   "история": {"days": ["вторник", "пятница"], "time": "16:00"},
   "математика": {"days": ["понедельник", "суббота"], "time": "17:00"},
   "русский язык": {"days": ["среда", "четверг"], "time": "18:00"},
}
```

1. Измените дни и время по нужному предмету.
2. Сохраните файл (Ctrl+S).
3. Перезапустите бота: закройте чёрное окно и снова запустите командой python bot.py.

⚠ **ВАЖНО:** Не удаляйте запятые после каждой строки и не меняйте структуру фигурных скобок!

1.2. Добавление нового курса

Чтобы добавить новый предмет (например, «Физика»):

1. В блоке SCHEDULE добавьте новую строку.
2. В блоке COURSE_SYNONYMS добавьте синонимы.
3. В блоке COURSES_INFO добавьте описание.
4. Сохраните файл и перезапустите бота.

РАЗДЕЛ 2. МОНИТОРИНГ ЛОГОВ

2.1. Где хранятся логи

Все взаимодействия пользователей сохраняются в файле bot_log.json в той же папке, где находится bot.py.

2.2. Как открыть логи

1. Кликните правой кнопкой по файлу bot_log.json.
2. Выберите «Открыть с помощью → Блокнот».
3. Вы увидите массив записей в формате JSON.

2.3. Структура одной записи

Каждая запись содержит:

- timestamp — дата и время сообщения
- session_id — идентификатор пользователя
- user_message — что написал пользователь
- intent — распознанное намерение бота
- is_success — true (успех) или false (ошибка)
- response_text — ответ бота
- confidence_score — уверенность бота (0.0–1.0)
- response_time_ms — время ответа в миллисекундах

2.4. Как рассчитать точность (Accuracy)

Запустите бота командой python bot.py. После завершения работы (команда «выход») бот автоматически выведет в консоль метрику Accuracy.

Целевой показатель: не менее 80%.

РАЗДЕЛ 3. АНАЛИЗ ОШИБОК

3.1. Как найти проблемные запросы

Откройте bot_log.json и используйте поиск (Ctrl+F) по слову: "is_success": false

3.2. Как улучшить распознавание

Если вы видите, что пользователи часто пишут одно и то же непонятное боту слово (например, «алгебра» вместо «математика»):

1. Откройте bot.py.
2. Найдите блок COURSE_SYNONYMS.
3. Добавьте новое слово в нужный список.
4. Сохраните и перезапустите бота.

РАЗДЕЛ 4. РЕШЕНИЕ ТИПОВЫХ ПРОБЛЕМ

Проблема: Ошибка «Permission denied: bot_log.json»
Решение: Закройте файл bot_log.json в Блокноте и перезапустите бота.

Проблема: Бот не запускается, ошибка «No such file»
Решение: Убедитесь, что вы находитесь в папке с файлом bot.py.

Проблема: Бот отвечает, но не запоминает данные записи
Решение: В текущей версии сессия хранится в памяти и сбрасывается при перезапуске бота.

РАЗДЕЛ 5. КОНТАКТЫ ТЕХПОДДЕРЖКИ

По всем техническим вопросам обращайтесь к разработчику:

- 👤 Пилигримова Екатерина Габделахатовна
- 📧 [ваш email]

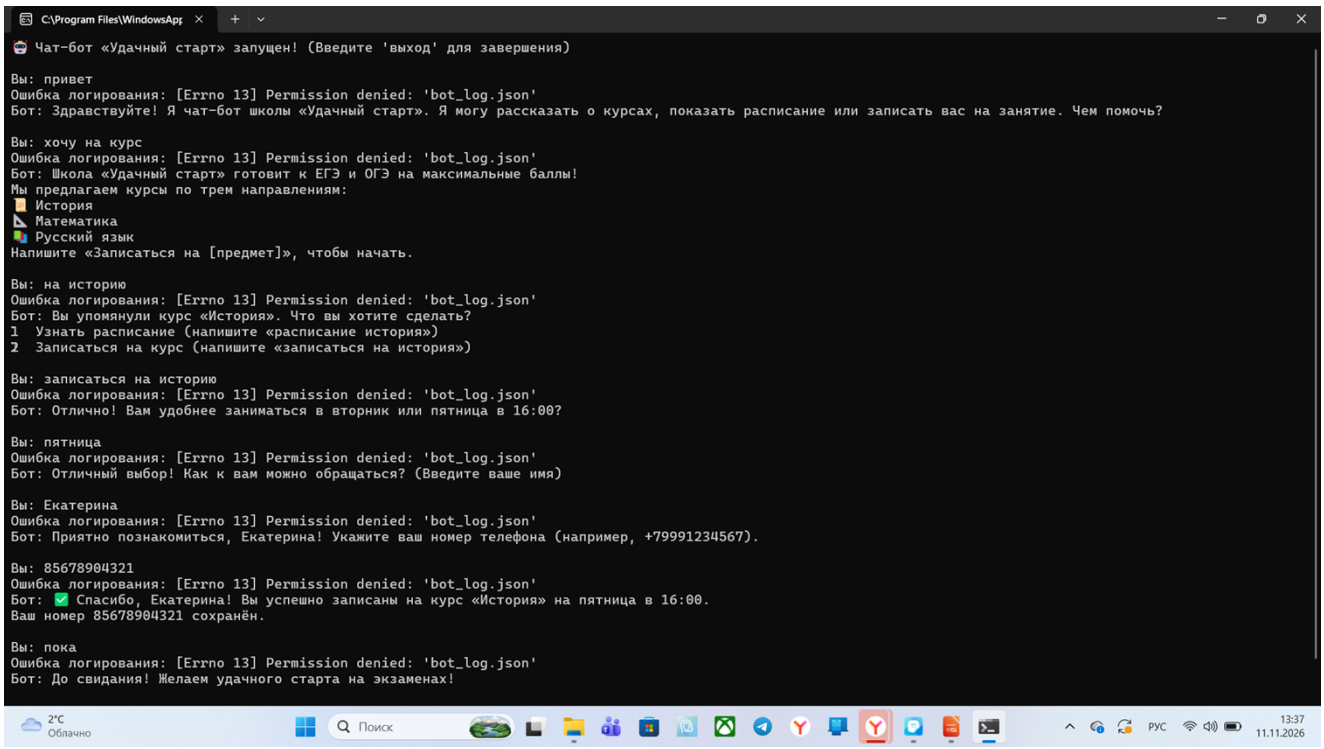
Приложение Г. Пример файла bot_log.json

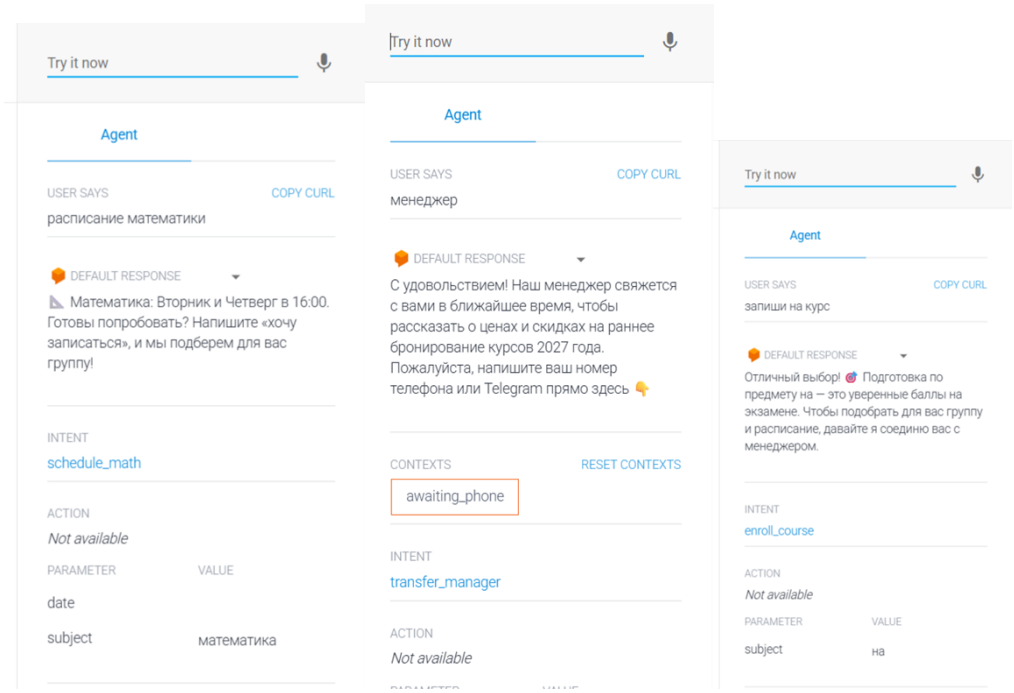
```
[
  {
    "timestamp": "2026-11-12T14:35:22.123456",
    "session_id": "test_user_001",
    "user_message": "Привет",
    "intent": "greet",
    "is_success": true,
    "response_text": "Здравствуйте! Я чат-бот школы «Удачный старт». Я могу рассказать о курсах, показать расписание или записать вас на занятие. Чем помочь?",
    "confidence_score": 0.99,
    "response_time_ms": 12
  },
  {
    "timestamp": "2026-11-12T14:35:45.234567",
    "session_id": "test_user_001",
    "user_message": "Запиши меня на историю",
    "intent": "register_course",
    "is_success": true,
    "response_text": "Отлично! Вам удобнее заниматься во вторник в 16:00 или в пятницу в 16:00?",
    "confidence_score": 0.95,
    "response_time_ms": 18
  },
  {
    "timestamp": "2026-11-12T14:36:10.345678",
    "session_id": "test_user_001",
    "user_message": "Во вторник",
    "intent": "provide_time_slot",
    "is_success": true,
```

"response_text": "Отличный выбор! Как к вам можно обращаться? (Введите ваше имя)",
"confidence_score": 0.95,
"response_time_ms": 15
},
{
"timestamp": "2026-11-12T14:36:35.456789",
"session_id": "test_user_001",
"user_message": "Анна",
"intent": "provide_name",
"is_success": true,
"response_text": "Приятно познакомиться, Анна! Укажите ваш номер телефона (например, +79991234567).",
"confidence_score": 0.95,
"response_time_ms": 14
},
{
"timestamp": "2026-11-12T14:37:02.567890",
"session_id": "test_user_001",
"user_message": "+79991234567",
"intent": "registration_complete",
"is_success": true,
"response_text": "✅ Спасибо, Анна! Вы успешно записаны на курс «История» на вторник в 16:00. Ваш номер +79991234567 сохранён.",
"confidence_score": 1.0,
"response_time_ms": 20
},
{
"timestamp": "2026-11-12T14:37:30.678901",
"session_id": "test_user_001",

```
  "user_message": "Спасибо",
  "intent": "thank_you",
  "is_success": true,
  "response_text": "Рад был помочь!",
  "confidence_score": 0.99,
  "response_time_ms": 10
 },
 {
  "timestamp": "2026-11-12T14:38:15.789012",
  "session_id": "test_user_002",
  "user_message": "Какая погода на завтра?",
  "intent": "unknown",
  "is_success": false,
  "response_text": "Извините, я не совсем понял ваш запрос.  Чтобы решить сложный вопрос, я могу переключить вас на живого менеджера. Напишите «менеджер», если хотите.",
  "confidence_score": 0.0,
  "response_time_ms": 11
 }
]
```


Приложение Д. Скриншоты работы бота


C:\Program Files\WindowsApp
Чат-бот «Удачный старт» запущен! (Введите 'выход' для завершения)
Вы: привет
Ошибка логирования: [Errno 13] Permission denied: 'bot_log.json'
Бот: Здравствуйте! Я чат-бот школы «Удачный старт». Я могу рассказать о курсах, показать расписание или записать вас на занятие. Чем помочь?
Вы: хочу на курс
Ошибка логирования: [Errno 13] Permission denied: 'bot_log.json'
Бот: Школа «Удачный старт» готовит к ЕГЭ и ОГЭ на максимальные баллы!
Мы предлагаем курсы по трем направлениям:
История
Математика
Русский язык
Напишите «Записаться на [предмет]», чтобы начать.
Вы: на историю
Ошибка логирования: [Errno 13] Permission denied: 'bot_log.json'
Бот: Вы упомянули курс «История». Что вы хотите сделать?
1 Узнать расписание (напишите «расписание история»)
2 Записаться на курс (напишите «записаться на история»)
Вы: записаться на историю
Ошибка логирования: [Errno 13] Permission denied: 'bot_log.json'
Бот: Отлично! Вам удобнее заниматься в вторник или пятница в 16:00?
Вы: пятница
Ошибка логирования: [Errno 13] Permission denied: 'bot_log.json'
Бот: Отличный выбор! Как к вам можно обращаться? (Введите ваше имя)
Вы: Екатерина
Ошибка логирования: [Errno 13] Permission denied: 'bot_log.json'
Бот: Приятно познакомиться, Екатерина! Укажите ваш номер телефона (например, +79991234567).
Вы: 85678904321
Ошибка логирования: [Errno 13] Permission denied: 'bot_log.json'
Бот: Спасибо, Екатерина! Вы успешно записаны на курс «История» на пятница в 16:00.
Ваш номер 85678904321 сохранён.
Вы: пока
Ошибка логирования: [Errno 13] Permission denied: 'bot_log.json'
Бот: До свидания! Желаем удачного старта на экзаменах!
2°C
Облачно
Поиск
РУС
13:37
11.11.2026


Try it now
Agent
USER SAYS
COPY CURL
расписание математики
DEFAULT RESPONSE
Математика: Вторник и Четверг в 16:00. Готовы попробовать? Напишите «хочу записаться», и мы подберем для вас группу!
INTENT
schedule_math
ACTION
Not available
PARAMETER
VALUE
date
subject
математика
Try it now
Agent
USER SAYS
COPY CURL
менеджер
DEFAULT RESPONSE
С удовольствием! Наш менеджер свяжется с вами в ближайшее время, чтобы рассказать о ценах и скидках на раннее бронирование курсов 2027 года. Пожалуйста, напишите ваш номер телефона или Telegram прямо здесь
CONTEXTS
RESET CONTEXTS
awaiting_phone
INTENT
transfer_manager
ACTION
Not available
Try it now
Agent
USER SAYS
COPY CURL
запиши на курс
DEFAULT RESPONSE
Отличный выбор! Подготовка по предмету на — это уверенные баллы на экзамене. Чтобы подобрать для вас группу и расписание, давайте я соединю вас с менеджером.
INTENT
enroll_course
ACTION
Not available
PARAMETER
VALUE
subject
на